BITACORA – LAB07

Alumno: Sanchez Mayen Tristan Qesen

⇒ **Paso 1 · La historia - En tus palabras: qué se rompió al pasar de llamada a método a HTTP.**

El problema con haber pasado de un monolito a cinco sistemas diferentes, es que la comunicación entre ellos puede complicar los procesos. En un sistema monolitico, se realiza el proceso de forma lineal y en el mismo sistema, por lo tanto, si algo falla, el mismo sistema lo detectara y podra deshacer una operación, mientras que al separarlo en cinco servicios, si el primero de ellos realiza bien el proceso que le corresponde, dara como completado dicho proceso, sin embargo, este proceso puede fallar en servicios posteriores y el primer servicio no se dara cuenta. Por eso es que digo que la comunicación entre los servicios podra generar problemas.

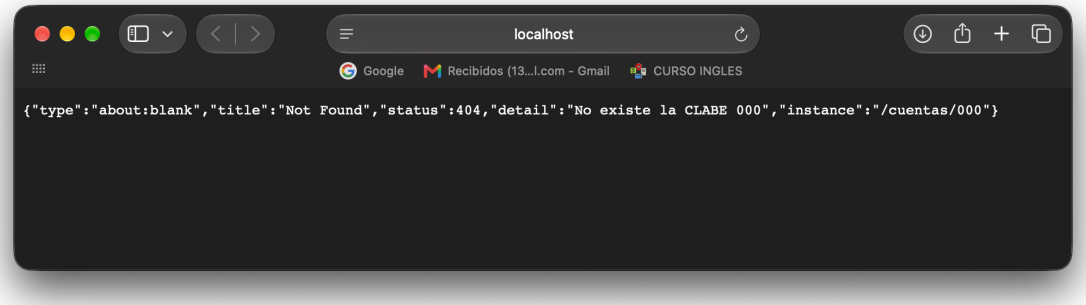
⇒ **Paso 2 · Directorio - El tablero antes (no carga) y después (cinco servicios UP), el XML de CUENTA y tu respuesta al "Piensa".**

Antes de comenzar a realizar cambios en el proyecto, es importante evidenciar el estado actual del mismo. Para ello utilizaremos los comandos:

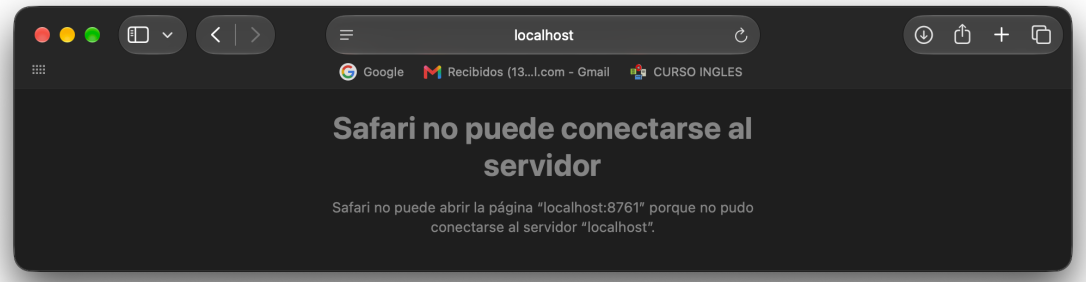
```
docker compose up -d     # levanta lo que ya está construido
docker compose ps        # seis contenedores: la base y los cinco servicios, cada uno con su puerto
```

Esto levantara los contenedores, sin embargo, al visitar los servicios, notaremos que:

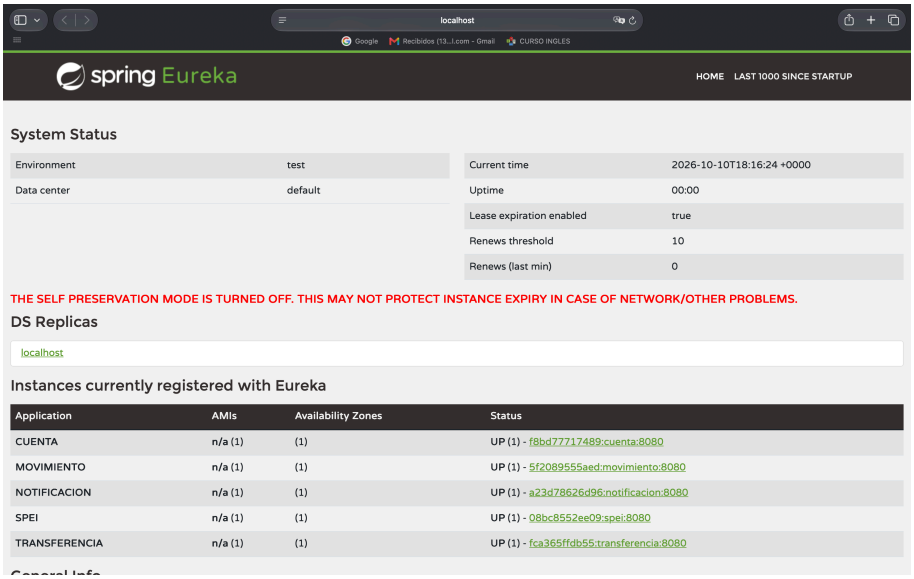
- http://localhost:8081/cuentas/000 → No encuentra la cuenta 000

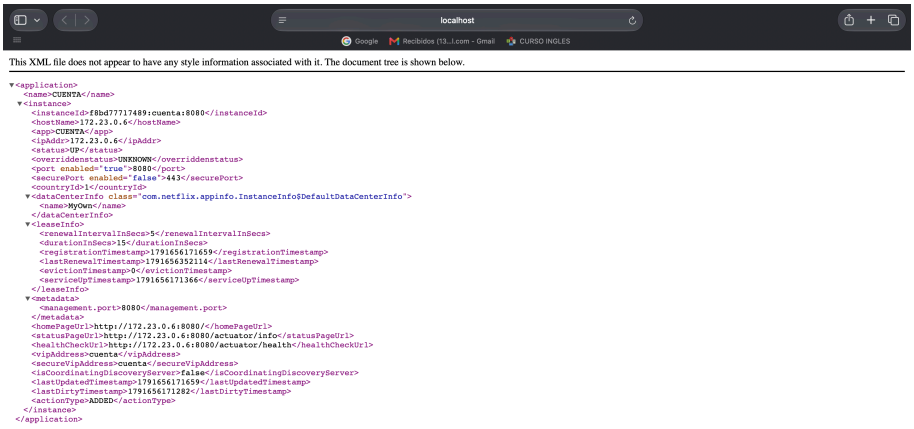


- http://localhost:8761 → No carga



Para solucionarlo es necesario que configuremos Eureka, una vez hecho, en http://localhost:8761 se mostraran los cinco servicios con un estatus de "UP":





Aunque los cinco servicios ya están anotados en el directorio de Eureka, nadie lo está usando todavía. El servicio de transferencia sigue buscando a la cuenta mediante la

dirección estática http://cuenta:8080 definida en el docker-compose, por lo que el directorio no está resolviendo el enrutamiento interno aún.

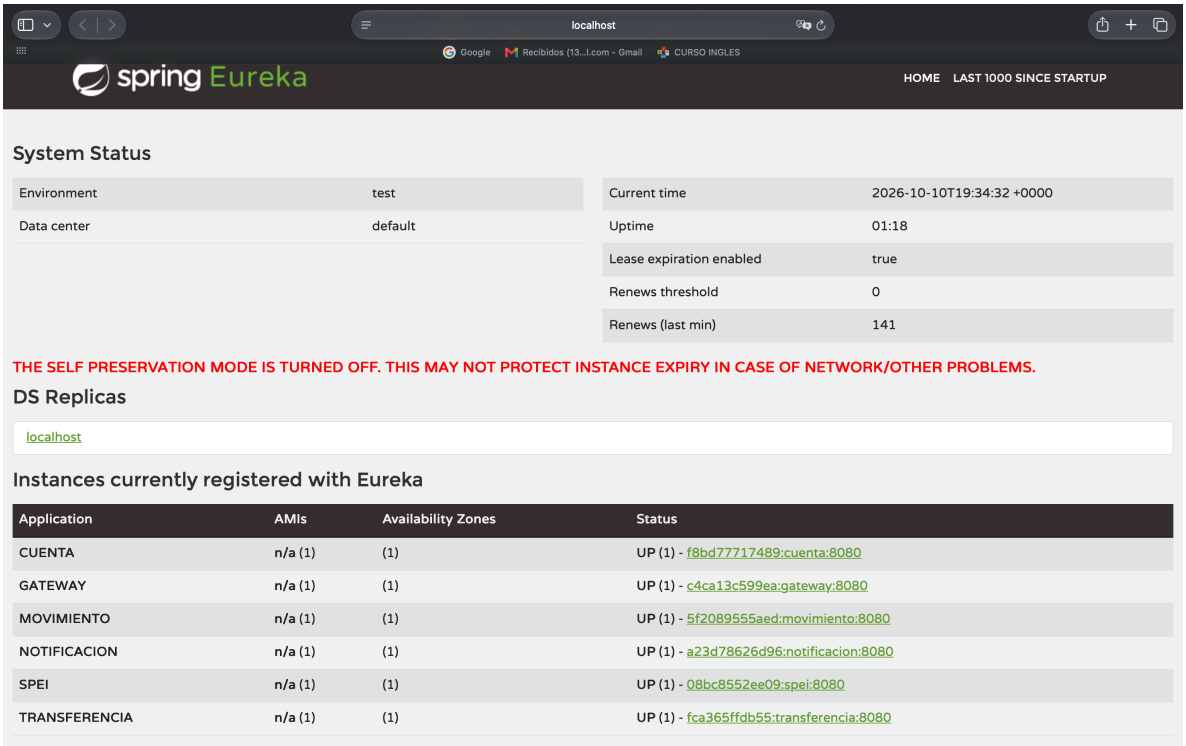
⇒ Paso 3 · Gateway - El 400 del 8082, los dos 404 por el gateway y quién produjo cada uno, el alta 201 y la consulta 200.

Para comprobar como funciona un Gateway, primero debemos de ver que es lo que soluciona. Al ejecutar el comando *curl -i -X POST http://localhost:8082/movimientos* nos arroja un error 400:

```
qesensinu@MacBook-Air-de-Tristan mexi-banco % curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 18:30:07 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

Para solucionar esto debemos de configurar el Gateway. Una vez configurado, el gateway tambien saldra en los Eureka:



Y cuando ejecutamos el comando curl para hacer una llamada al sistema, ya no sale el error 400, ahora sale el 404.

Ya que sabemos que el servicio responde, podemos crear una cuenta de prueba:

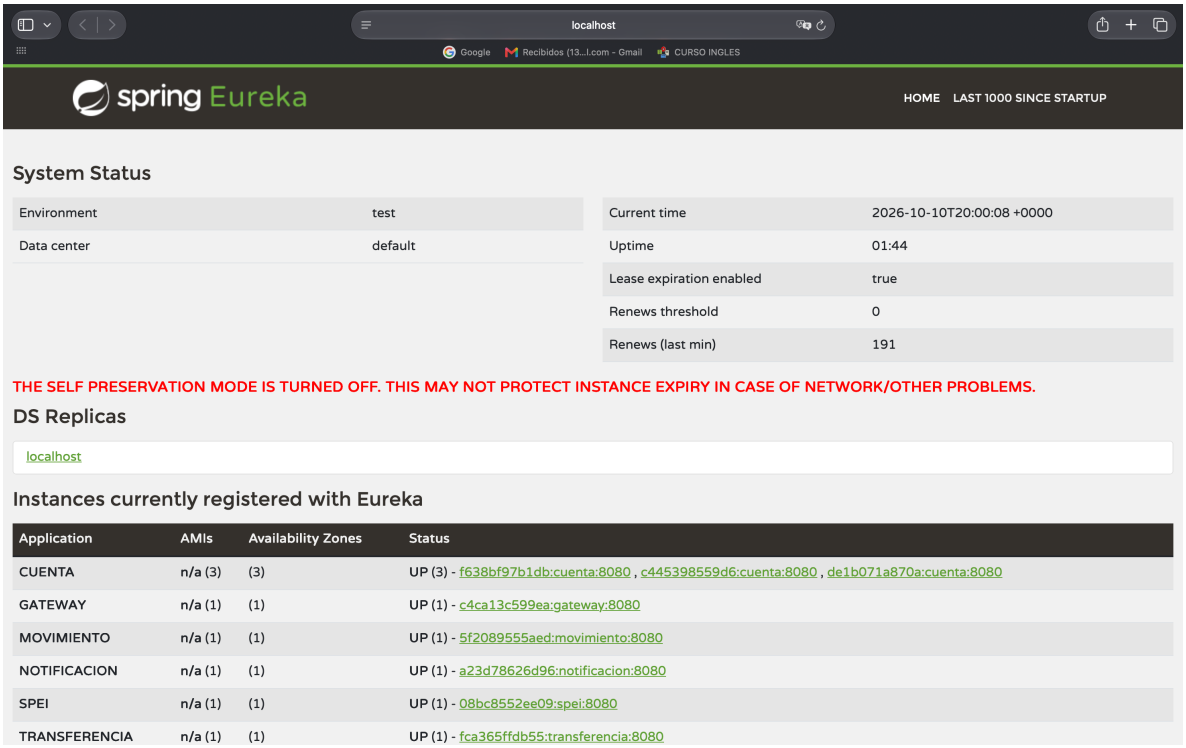


⇒ **Pasos 4 y 5 · Copias y caída - El tablero con CUENTA UP (3), las nueve X-Instancia, tu predicción, la hora del apagado, la de la salida del directorio, cuántos errores viste y por qué.**

Ahora crearemos tres copias de ‘cuenta’, para ello se ajusta en el docker-compose y luego se vuelve a levantar el contenedor. Por ultimo para verificarlo usaremos el siguiente comando que nos muestra las copias de cuenta:

```
qesensinu@MacBook-Air-de-Tristan mexi-banco % docker ps --filter name=cuenta --format '{{.ID}}  {{.Names}}  {{.Ports}}'
de1b071a870a  mexi-banco-v06-cuenta-1  8080/tcp
f638bf97b1db  mexi-banco-v06-cuenta-2  8080/tcp
c445398559d6  mexi-banco-v06-cuenta-3  8080/tcp
qesensinu@MacBook-Air-de-Tristan mexi-banco %
```

Si entramos a http://localhost:8761/ veremos que cuenta tiene un (3), este numero significa el numero de copias de este servicio



Para saber como es que Eureka reparte la entrada de transiciones usamos un ciclo for e imprimimos la cabecera que dice qué copia contestó:

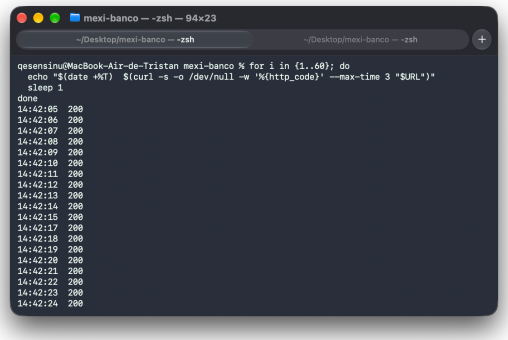
```
qesensinu@MacBook-Air-de-Tristan mexi-banco % for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: f638bf97b1db
X-Instancia: de1b071a870a
X-Instancia: c445398559d6
X-Instancia: f638bf97b1db
X-Instancia: de1b071a870a
X-Instancia: c445398559d6
X-Instancia: f638bf97b1db
X-Instancia: de1b071a870a
X-Instancia: c445398559d6
qesensinu@MacBook-Air-de-Tristan mexi-banco %
```

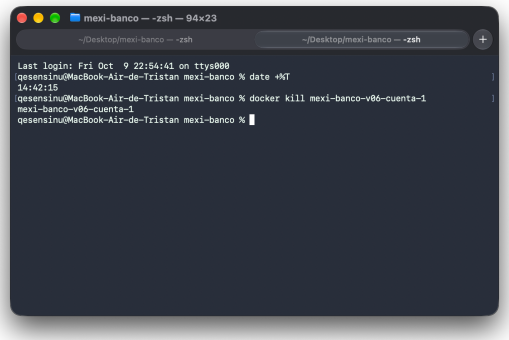
Podemos ver que la primera transaccion la hara a la primera copia, la segunda a la segunda copia y la tercera a la tercera copia. Despues regresa a la primera copia y asi sucesivamente.

¿Van a fallar todas las consultas, ninguna o algunas? ¿Durante cuánto tiempo? ¿Dejan de fallar en cuanto la copia desaparece del directorio, o antes, o después?

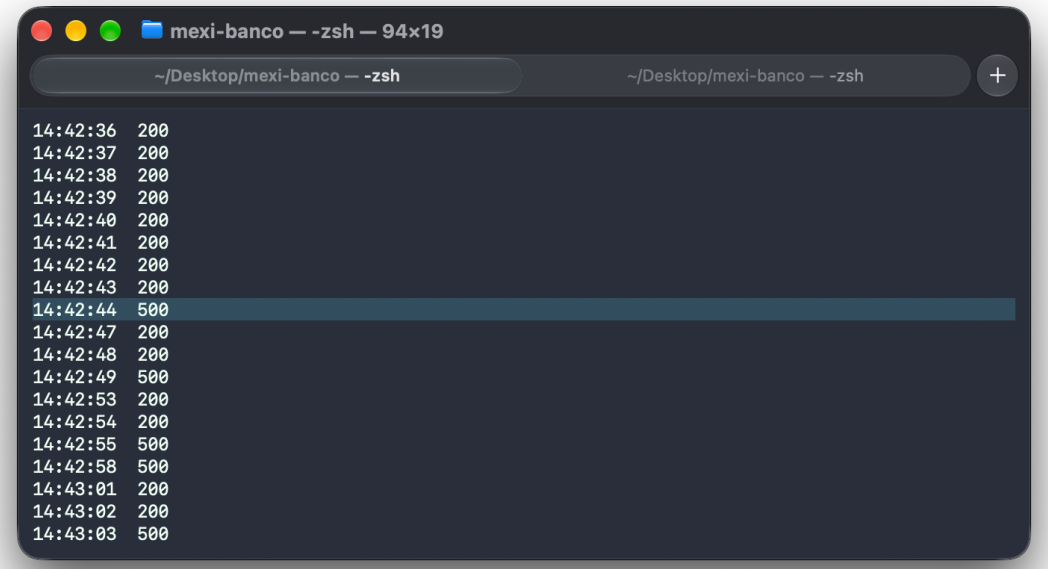
Al apagar una copia de golpe, algunas peticiones van a fallar temporalmente, pero el sistema se recuperará después de unos segundos.

Al hacer la prueba de eliminar una copia a modo de simulacion de que esta copia se cayo. , para esto se usan dos terminales, una donde se muestre una consulta cada segundo y en la otra haremos la eliminacion de una de las copias.





En la primera terminal se observa el tiempo en el que se estuvo registrando las consultas y en la segunda el tiempo en el que se elimino la copia. Podemos ver que todas las consultas arrojan un cod



Application	AMIs	Availability Zones	Status
CUENTA	n/a (2)	(2)	UP (2) - f638bf97b1db:cuenta:8080 , c445398559d6:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - c4ca13c599ea:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 5f2089555aed:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - a23d78626d96:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 08bc8552ee09:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - fca365ffdb55:transferencia:8080

En la anterior captura podemos ver el momento en el que se comenzo a dar el codigo 500 de error. Durante este proceso, se dieron 6 codigos 500 en 66 segundos. Ademas, la copia salio del directorio a los 53 segundos de haber eliminado la copia.

Para poder revivir la copia ejecutaremos el comando docker start mexi-banco-v06-cuenta-1, el cual devolvera el (3) en el Eureka.

⇒ Cierre - Tus cuatro frases.

El **directorio** resuelve saber qué servicios hay y dónde están, sin necesidad de escribir sus direcciones IP o puertos a mano en el código. El **gateway** resuelve tener una sola puerta de entrada segura para el cliente, estableciendo reglas sobre qué operaciones se pueden llamar desde afuera y ocultando la red interna. El **balanceador** resuelve repartir la carga de trabajo entre múltiples copias de un mismo servicio y dejar de mandar peticiones a las copias que mueren. **Cuando murió una copia** vi errores 500 intermitentes durante unos 29 segundos porque el sistema depende de la ausencia de *heartbeats* (que toman 15s de plazo) y del tiempo de refresco de la caché del gateway.

⇒ Tarea - Captura de ./demo-v06.sh con los saldos.

Al ejecutar el script podemos ver el proceso que se realizo, pero para evitar tecnisismos, podemos ir directamente a los saldos. Primero la transferencia que se realiza de Ana a Beto.

```
=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009167181501
{
  "id": 2,
  "clabe": "002180009167181501",
  "titular": "Ana",
  "saldo": 800.00
}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009167181502
{
  "id": 3,
  "clabe": "002180009167181502",
  "titular": "Beto",
  "saldo": 700.00
}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700
```

Despues se hace otra transferencia para probar la idempotencia (realizar la misma operación dos veces), a pesar de provocar un error, solo se mantendra una sola transferencia, danto solo una resta de -50 a Ana.

```
=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco
```

BITACORA – LAB07 (Corrección)

Alumno: Sanchez Mayen Tristan Qesen

⇒ Corrección solicitada: Los dos errores 404 del Gateway y su origen

Observación de la revisión (Corte 4-nov-2026)

"S07: los dos 404 del gateway no dicen quién produjo cada uno. Cómo se corrige: Muestra el GET a /api/cuentas/000 (lo contesta cuenta, trae X-Instancia) y el POST a /api/movimientos (lo contesta el gateway, sin X-Instancia) y explica la diferencia."

1. Petición: `GET http://localhost:8080/api/cuentas/000`

```
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 19:42:15 GMT
X-Instancia: f638bf97b1db
transfer-encoding: chunked
{"type":."about:blank","title":"Not Found","status":404,
  "detail":"No existe la CLABE 000",ïnstance":/cuentas/000"}
```

¿Quién lo produjo?: **El microservicio** `cuenta`.

Explicación: El API Gateway tiene configurada la ruta `/api/cuentas/**` hacia el servicio `cuenta`. Al recibir esta petición por el puerto 8080, el Gateway consultó a Eureka, resolvió la IP/puerto de una réplica viva de `cuenta`, removió el prefijo (`StripPrefix=1`) y le transfirió la llamada como `GET /cuentas/000`. El servicio `cuenta` procesó la búsqueda en su base de datos y, al no encontrar registros para la CLABE `000`, generó la respuesta 404 en formato JSON RFC 7807 (`detail: "No existe la CLABE 000"`). Al viajar la respuesta de regreso, se incluye la cabecera `X-Instancia: f638bf97b1db`, la cual comprueba que la petición efectivamente cruzó la frontera del Gateway y fue contestada por un contenedor específico de `cuenta`.

2. Petición: `POST http://localhost:8080/api/movimientos`

```
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
content-length: 0
date: Sat, 10 Oct 2026 19:42:25 GMT
```

¿Quién lo produjo?: **El API Gateway**.

Explicación: En la configuración de rutas del Gateway (`application.yml`), la ruta `/api/movimientos` únicamente está declarada para el método HTTP `GET` (consultas públicas del historial de movimientos). Registrar un movimiento (`POST`) es una operación contable interna sensible que no debe estar expuesta a clientes externos, sino únicamente a otros microservicios en la red interna Docker. Al recibir un `POST`, el Gateway evalúa sus predicados de ruta, no encuentra ninguna coincidencia, y frena la llamada inmediatamente en el perímetro devolviendo un 404 genérico. La petición **jamás llega al servicio** `movimiento` ni se consulta a Eureka; por esta razón, la respuesta **NO incluye la cabecera** `X-Instancia`.

3. La diferencia fundamental

Ambos comandos devuelven el código HTTP 404, pero representan naturalezas completamente distintas:

- En `GET /api/cuentas/000` es un **404 de negocio**: la ruta sí existe en el Gateway, el balanceo se ejecutó, el tráfico llegó al microservicio de fondo y fue éste quien respondió que el dato no existe (confirmado por `X-Instancia`).
- En `POST /api/movimientos` es un **404 de infraestructura / seguridad perimetral**: el Gateway actuó como firewall de aplicación rechazando una operación no autorizada sin tocar la red interna (confirmado por la ausencia de `X-Instancia`).